

Про курс



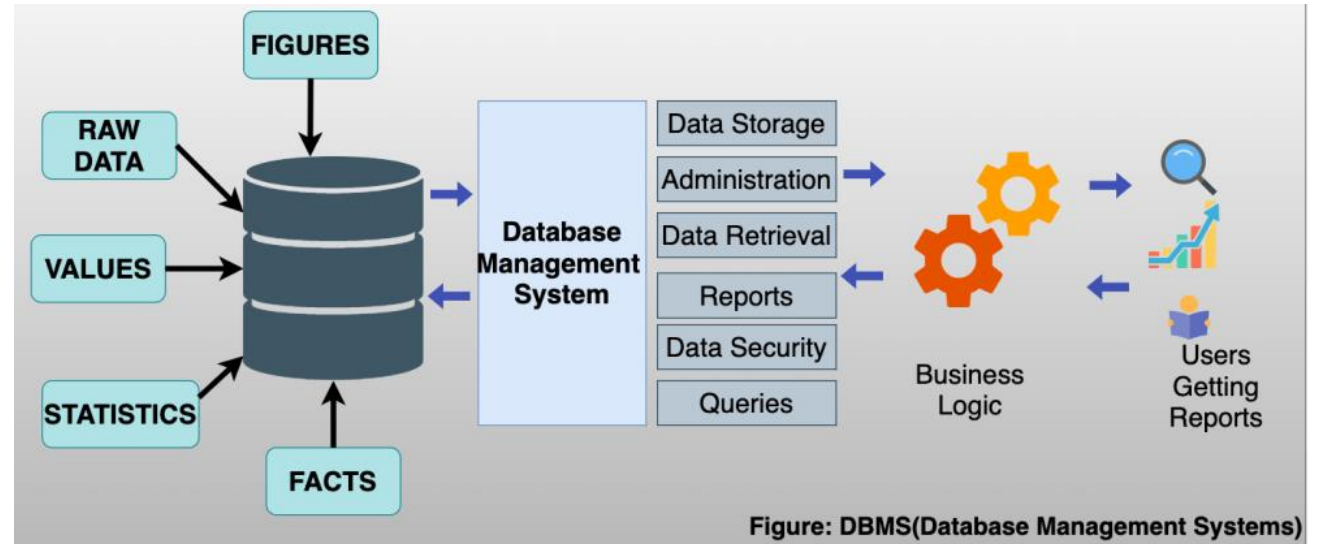
- ✓ Вступ до NoSQL
- ✓ Документо-орієнтовані бази даних (MongoDB)
- ✓ Ключ-значення бази даних (Redis)
- ✓ Графові бази даних (Neo4j)
- ✓ Колонкові бази даних (Cassandra)

Що таке БАЗИ ДАНИХ та СУБД?

База даних — це організована електронна колекція структурованої інформації або даних, яка зберігається в електронному форматі в комп'ютерній системі.

Система управління базами даних (СУБД) — це програмне забезпечення для обробки, зміни та отримання даних у базі. Вона маніпулює даними, їх форматом, назвами полів і структурою записів, а також визначає правила для валідації та маніпуляції даними.

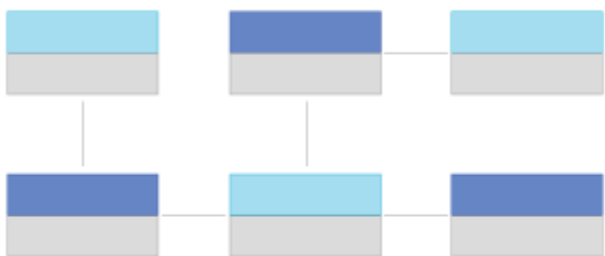
СУБД взаємодіє з користувачами, додатками та базою даних для збору та аналізу даних, дозволяючи визначати, створювати, запитувати, оновлювати та адмініструвати бази даних.



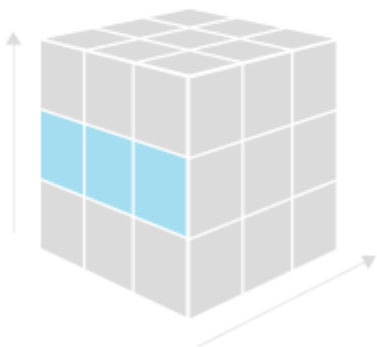
Реляційні бази даних (SQL)

SQL

Relational



Analytical



Реляційна база даних — це база, де дані структуруються у вигляді таблиць із рядками (записами) та стовпцями (полями).

Між таблицями встановлюються чітко визначені взаємозв'язки.

Схема даних суворо визначена заздалегідь.

Використовують структуровану мову запитів **SQL**.

Ключові переваги — цілісність, узгодженість та надійність транзакцій.

Приклади СУБД: MySQL, PostgreSQL, MariaDB, Oracle та Microsoft SQL Server.

```
1 SELECT country, sum(customer_count) total_customers, sum(spend) ad_spend
2 FROM (
3   SELECT customers.ip_country, count(email) customer_count
4   FROM customers
5   WHERE customers.createddate > '2020-01-01'
6   GROUP BY country) new_customers
7 JOIN facebook_ads ON facebook_ads.country = new_customers.ip_country
8 WHERE ad_spend.date > '2020-01-01'
9 GROUP BY country
10 ORDER BY ad_spend desc;
```

id	name	type	size
1	Rex	dog	big
2	Cleo	dog	small
3	Leroy	dog	medium
4	Sunny	cat	big
5	Snow Ball	cat	medium

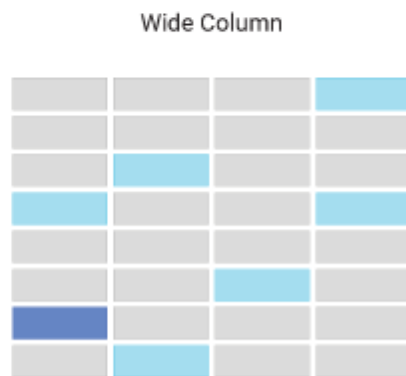
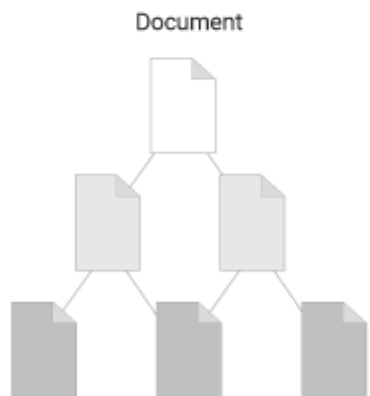
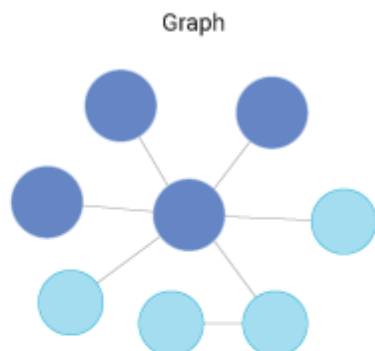
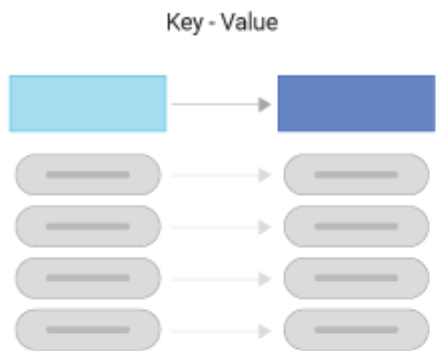
id_pet	weight	age
1	32,3	7
2	4,35	13,6
3	24,9	4
5	8,75	5

name	food	danger_level
Cleo	diet	5
Sunny	wet	1
Rex	dry	4
Leroy	wet	3



Нереляційні бази даних (NoSQL)

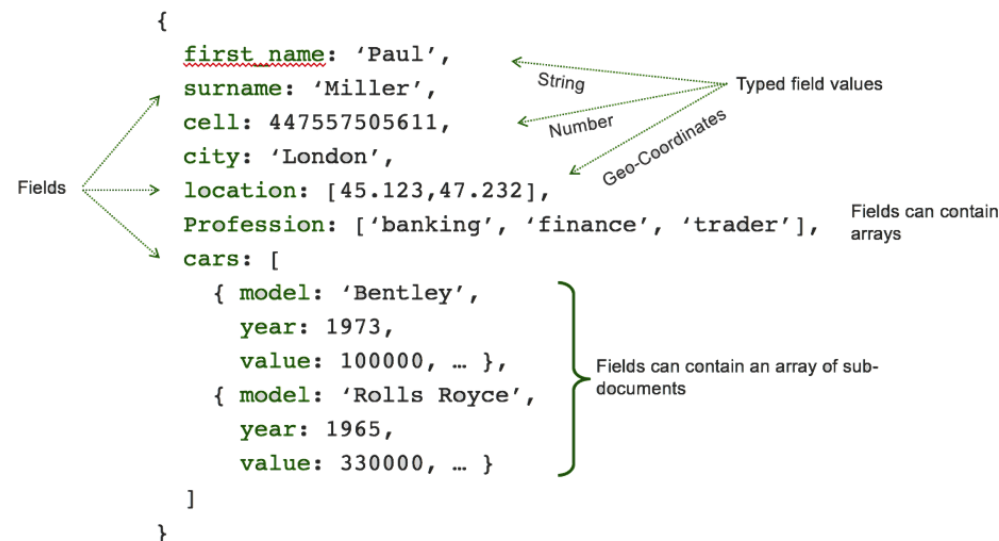
NoSQL



Нереляційні бази даних, також відомі як **NoSQL** бази даних, зберігають дані у форматах, відмінних від таблиць.

Вони призначені для обробки великих обсягів даних і є гнучкими щодо моделі даних.

NoSQL — це клас систем управління базами даних (DBMS), які не використовують SQL і не дотримуються реляційної моделі.



mongoDB®



redis

neo4j



cassandra

Що спільного у додатків у вашому смартфоні?



Мільйони дій щосекунди: замовлення таксі (Uber, Bolt), покупки (Rozetka, AliExpress), стрімінг медіа (Netflix, Spotify).

Спільний знаменник: кожен із цих сервісів працює на базі NoSQL або гібридної архітектури (**Polyglot Persistence**).

Чому не класичний SQL? Масштаб персоналізації та швидкість відгуку вимагають відмови від монолітної структури.

БАЗИ ДАНИХ

Relational
Database

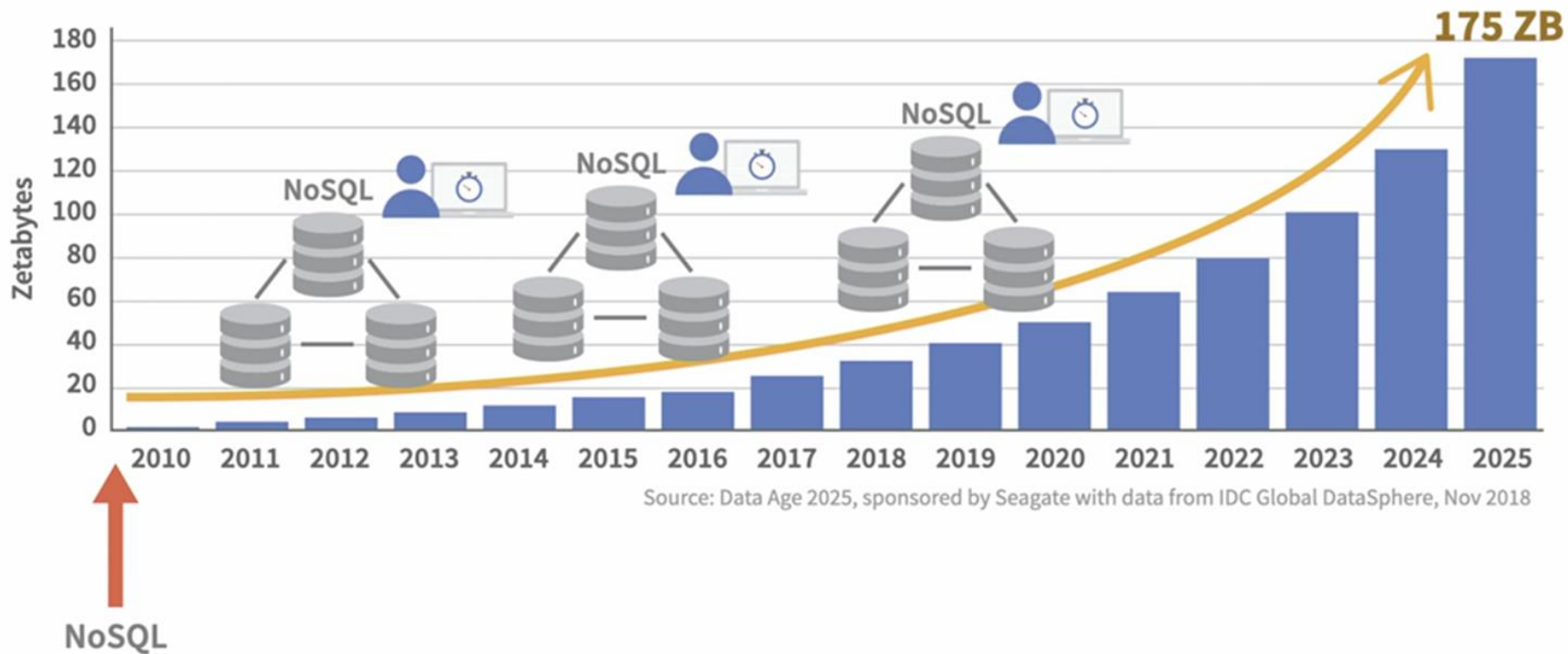


NoSQL
Database



Передумови змін: Стрімке зростання даних

Annual Size of the Global Datasphere



Стрімкий розвиток Web 2.0, мобільних застосунків, соцмереж та IoT створив безпрецедентний потік даних. Обсяг світових даних зріс від кількох зеттабайтів у 2010 році до понад 175–180 ZB у 2025–2026 роках. Головний виклик: класичні монолітні реляційні сервери вперлися у фізичні та фінансові межі масштабування.

Великі дані

BIG DATA



Великі дані (Big Data) — набір структурованих, неструктурованих і напівструктурованих даних великих обсягів і значної різноманітності, що ефективно обробляються горизонтально масштабованими програмними інструментами.

Це не просто величезні масиви чисел, а скоріше новий спосіб мислення про інформацію, який відкриває безліч можливостей для бізнесу, науки та суспільства в цілому.

NoSQL не є синонімом великих даних

Великі дані: різноманітність форматів даних

Структуровані дані

Це дані, які мають чітко визначену структуру, тобто вони організовані в таблиці, де кожен рядок представляє запис, а кожен стовпець – атрибут. Ці дані легко вводити, зберігати та аналізувати за допомогою традиційних баз даних.

Приклади:

- Таблиці в базах даних (наприклад, таблиця клієнтів з полями "Ім'я", "Прізвище", "Адреса", "Телефон")
- Дані в електронних таблицях (Excel)
- Числові дані з датчиків

Employee_ID	Employee_Name	Gender	Department	Salary_In_lacs
2365	Rajesh Kulkarni	Male	Finance	650000
3398	Pratibha Joshi	Female	Admin	650000
7465	Shushil Roy	Male	Admin	500000
7500	Shubhojit Das	Male	Finance	500000
7699	Priya Sane	Female	Finance	550000

Напівструктуровані дані

Ці дані мають певну структуру, але вона не так жорстка, як у структурованих даних. Вони можуть містити теги або інші елементи, які допомагають визначити їх структуру.

Приклади:

- XML-файли
- JSON-файли
- Електронні листи
- Документи Word або PDF (хоча основний текст може бути неструктурованим, метадані про документ, такі як автор, дата створення, заголовки, можуть бути структурованими)

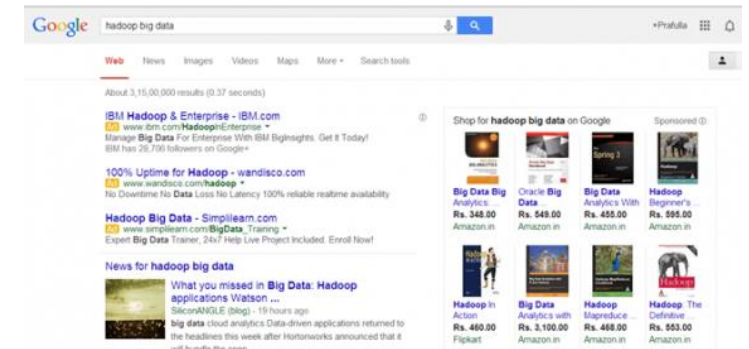
```
<rec><name>Prashant Rao</name><gender>Male</sex><age>35</age></rec>
<rec><name>Seema R.</name><gender>Female</sex><age>41</age></rec>
<rec><name>Satish Mane</name><gender>Male</sex><age>29</age></rec>
<rec><name>Subrato Roy</name><gender>Male</sex><age>26</age></rec>
<rec><name>Jeremiah J.</name><gender>Male</sex><age>35</age></rec>
```

Неструктуровані дані

Це дані, які не мають чітко визначеної структури. Вони можуть бути у вигляді тексту, зображень, аудіо або відео.

Приклади:

- Тексти документів (без форматування)
- Соціальні мережі (пости, коментарі)
- Зображення
- Аудіозаписи
- Відео



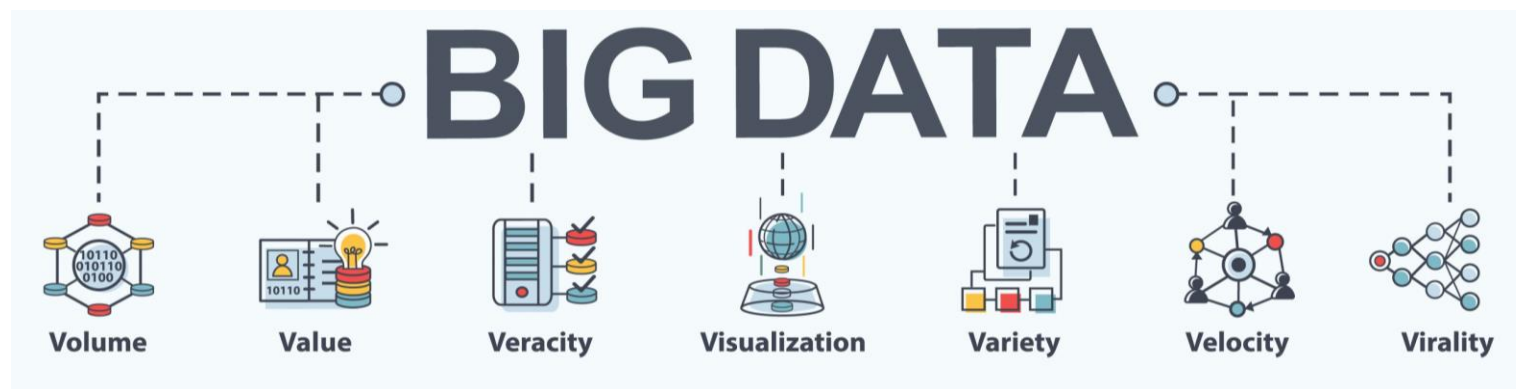
Сучасні додатки на 80% оперують саме напівструктурованими та неструктурованими даними

Великі дані: характеристики

Існує сім важливих характеристик великих даних, або «7 V»:

1. **Volume (обсяг):** Дані швидко зростають, досягаючи терабайтів і петабайтів. У 2025 році ~160-200 зетабайт даних.
2. **Velocity (швидкість):** Дані постійно оновлюються, наприклад, 2,5 млн запитів у Google щохвилини. Важливо аналізувати їх у реальному часі.
3. **Variety (різноманітність):** Дані мають різні формати — текст, графіка, відео, транзакції. Кожен формат потребує окремого підходу до аналізу.

4. **Value (цінність):** Важливо не лише зібрати дані, але й витягти з них корисну інформацію, яка може бути використана для прийняття рішень.
5. **Veracity (достовірність):** Дані мають бути достовірними для прийняття правильних рішень. Недостовірні дані можуть призвести до невдач.
6. **Variability (мінливість):** Дані можуть швидко змінювати контекст. Важливо адаптуватися до змін у поведінці цільової аудиторії.
7. **Visualization (візуалізація):** Якісна візуалізація робить дані зрозумілими. Діаграми, графіки та інтерактивні карти полегшують сприйняття.



Великі дані: користь

Великі дані (Big Data) мають широкий спектр застосувань у різних галузях. Деякі з основних напрямків:

1. Бізнес-аналітика та прийняття рішень:

- ❖ Оптимізація бізнес-процесів.
- ❖ Прогнозування попиту та управління запасами.
- ❖ Аналіз поведінки клієнтів для підвищення рівня обслуговування.

2. Маркетинг та реклама:

- ❖ Персоналізація рекламних кампаній.
- ❖ Аналіз ефективності маркетингових стратегій.
- ❖ Виявлення нових ринкових можливостей.

3. Охорона здоров'я:

- ❖ Аналіз медичних даних для покращення діагностики та лікування.
- ❖ Відстеження та прогнозування епідемій.
- ❖ Персоналізована медицина.

4. Фінансові послуги:

- ❖ Виявлення шахрайства.
- ❖ Управління ризиками.
- ❖ Аналіз ринкових тенденцій та інвестиційних можливостей.

5. Транспорт та логістика:

- ❖ Оптимізація маршрутів та управління транспортними засобами.
- ❖ Прогнозування попиту на транспортні послуги.
- ❖ Відстеження вантажів у реальному часі.

6. Енергетика:

- ❖ Управління енергоспоживанням.
- ❖ Прогнозування виробництва та споживання енергії.
- ❖ Виявлення несправностей у мережах.

7. Наука та дослідження:

- ❖ Аналіз великих обсягів наукових даних.
- ❖ Моделювання та симуляції.
- ❖ Виявлення нових наукових закономірностей.

8. Соціальні медіа та інтернет:

- ❖ Аналіз поведінки користувачів.
- ❖ Виявлення трендів та настроїв у суспільстві.
- ❖ Оптимізація контенту та рекомендаційних систем.

9. Управління містами (Smart Cities):

- ❖ Моніторинг та управління міською інфраструктурою.
- ❖ Оптимізація транспортних потоків.
- ❖ Підвищення безпеки та якості життя мешканців.

10. Освіта:

- ❖ Персоналізація навчальних програм.
- ❖ Аналіз успішності студентів.
- ❖ Виявлення та підтримка студентів, які потребують додаткової допомоги.

11. Штучний Інтелект

Великі дані: основні технології та інструменти

Етап роботи з Big Data	Технології / підходи	Що це означає
Збір даних	ETL / Stream Processing	ETL (Extract, Transform, Load) — вилучення, трансформація та завантаження даних у сховища. Stream Processing — обробка поточкових даних у реальному часі.
Зберігання	Hadoop / NoSQL / Data Warehouse	NoSQL — бази даних для зберігання та обробки великих обсягів неструктурованих даних. Data Warehouse — централізовані сховища для зберігання та аналізу великих обсягів інформації.
Обробка	Hadoop / Spark	Hadoop — платформа для розподіленої обробки великих даних. Spark — швидка платформа для обробки даних у пам'яті.
Аналіз	Data Mining / Machine Learning	Data Mining — методи видобування знань із великих масивів даних. Machine Learning — алгоритми машинного навчання для виявлення закономірностей.
Візуалізація	Data Visualization	Інструменти для візуального представлення та аналізу великих обсягів даних.
Інфраструктура	Cloud Platforms	Хмарні сервіси для зберігання, обробки та аналізу великих даних.

Великі дані – це потужний інструмент, який може змінити наш світ. Однак, для ефективного використання великих даних необхідно розуміти їх особливості та володіти відповідними технологіями.

NoSQL (Not Only SQL)

Термін NoSQL («not only SQL» або «не лише SQL») був уперше запропонований у 2009 році і застосовується до баз даних, у яких вирішуються проблема горизонтального масштабування та роботи з Big Data.

NoSQL (Not Only SQL) — категорія СУБД, орієнтованих на гнучке моделювання даних, масштабування та роботу з великими обсягами даних.

NoSQL бази даних зазвичай використовуються в ситуаціях, де реляційні бази даних можуть бути неефективними або складними у використанні.

NoSQL бази даних часто використовуються в сучасних веб-додатках, великих даних (Big Data) і реальних часах аналітики, де гнучкість, масштабованість і продуктивність є критично важливими.

Поява NoSQL не означає повну заміну реляційних баз даних. Сучасна архітектура програмного забезпечення часто керується принципом **багатоваріантної персистентності (Polyglot Persistence)**: використання різних типів СУБД для вирішення різних за специфікою задач у межах одного проєкту.

HOW TO WRITE A CV



Основні типи NoSQL баз даних

Документно-орієнтовані бази даних:

Зберігають дані у вигляді документів (наприклад, JSON, BSON).

Приклади: *MongoDB, CouchDB*.

Ключ-значення бази даних:

Зберігають дані у вигляді пар "ключ-значення".

Приклади: *Redis, BerkeleyDB, Riak*.

Колонкові бази даних:

Зберігають дані у вигляді стовпців, що дозволяє ефективно обробляти великі обсяги даних.

Приклади: *Apache Cassandra, HBase*.

Графові бази даних:

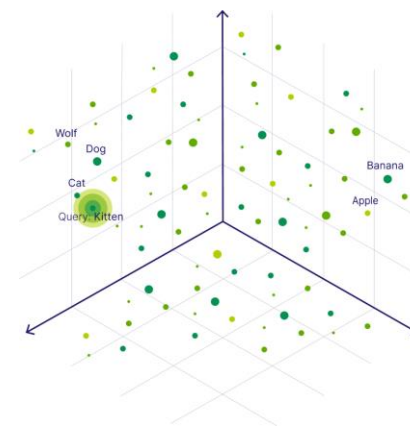
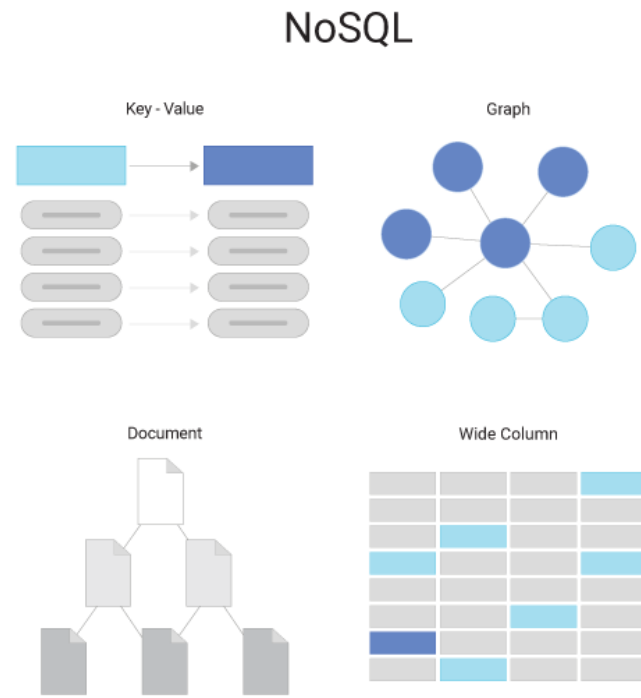
Оптимізовані для зберігання і обробки графів, де дані представлені у вигляді вузлів і ребер.

Приклади: *Neo4j, Orient db, ArangoDB*.

Векторні бази даних:

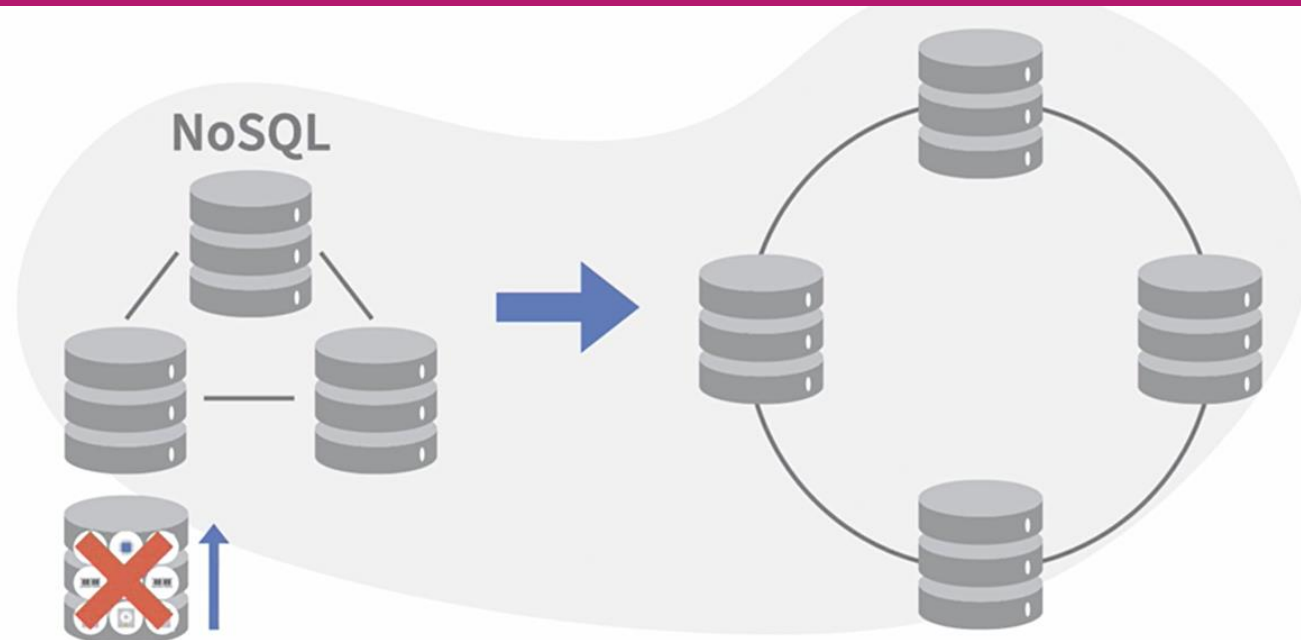
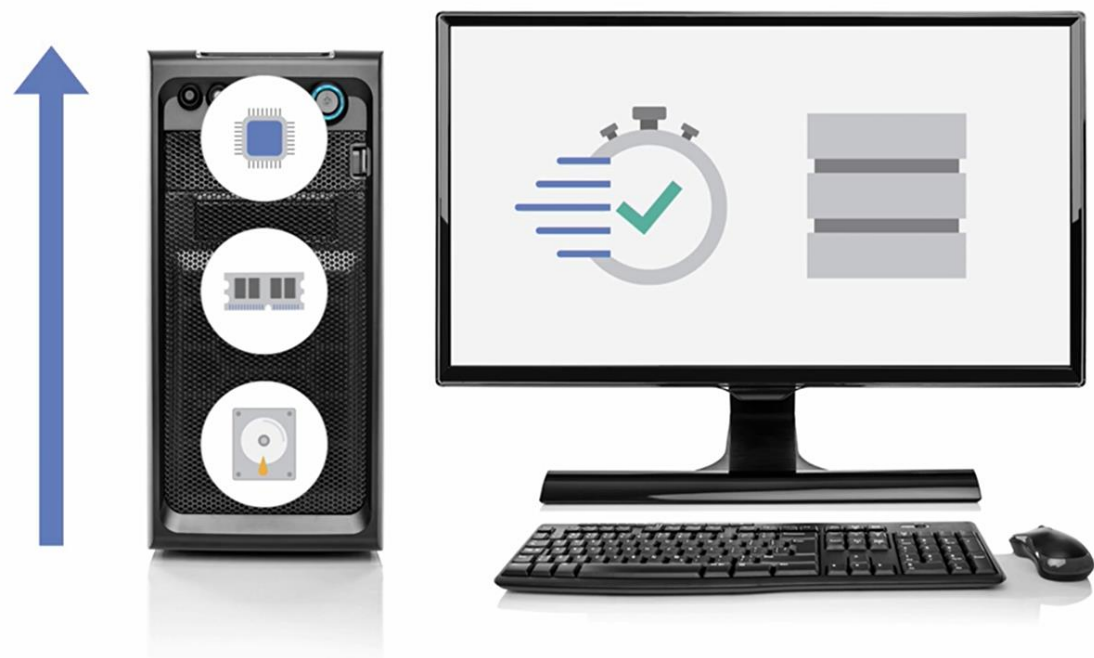
Оптимізовані для зберігання та семантичного пошуку багатовимірних векторів. AI, LLM та пошук за фото/змістом.

Приклади: *Pinecone, Milvus, Qdrant*.



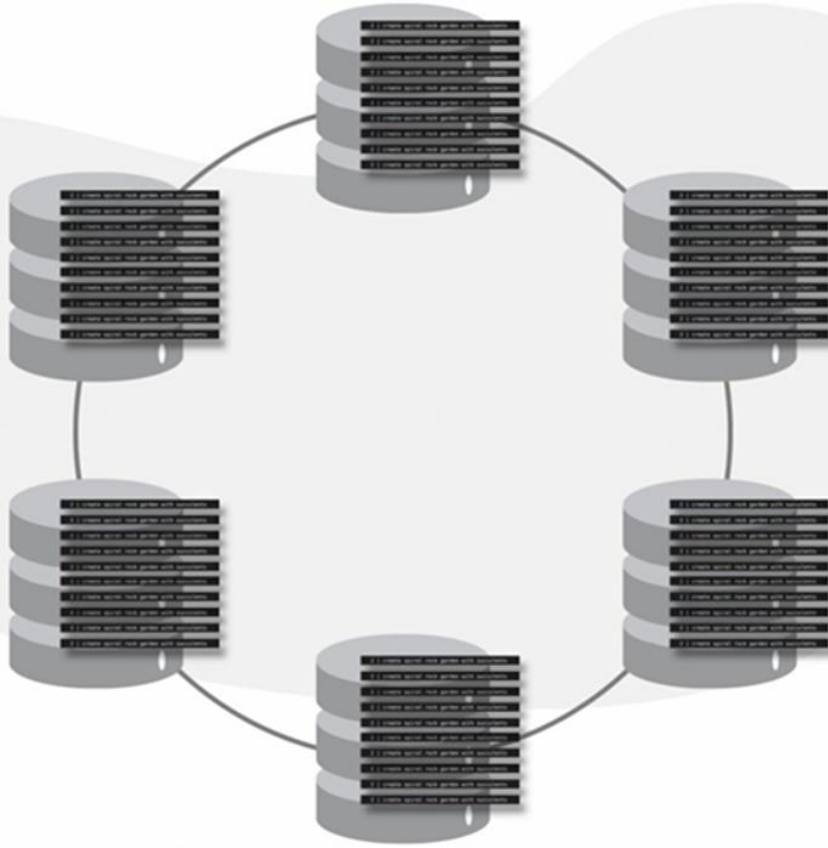
Вертикальне і горизонтальне масштабування

Вертикальне масштабування означає збільшення потужності одного комп'ютера або сервера шляхом додавання більш потужних компонентів, таких як процесор, оперативна пам'ять або дисковий простір. Це дозволяє одному серверу обробляти більше навантаження.



Горизонтальне масштабування означає додавання нових комп'ютерів або серверів до існуючої системи для розподілу навантаження. Це дозволяє системі обробляти більше запитів і зберігати більше даних за рахунок розподілу роботи між кількома машинами.

Горизонтальне масштабування: принцип роботи кластера



- ❖ **Розподіл навантаження:**
кожен сервер обробляє лише свою частину даних, що гарантує стабільну швидкість за будь-яких обсягів.
- ❖ **Еластичність на вимогу:**
нові сервери додаються до кластера «на ходу» без зупинки сервісу та простоїв.
- ❖ **Відмовостійкість без SPOF:**
«не кладемо всі яйця в один кошик» — збій одного сервера в кільці не зупиняє роботу всієї системи.

Вертикальне і горизонтальне масштабування

Вертикальне масштабування

Переваги:

- Простота реалізації.
- Не потребує змін у програмному забезпеченні.

Недоліки:

- Обмежена масштабованість (фізичні обмеження апаратного забезпечення).
- Висока вартість потужного обладнання.
- Єдиний точковий збій (single point of failure).

Горизонтальне масштабування

Переваги:

- Висока масштабованість (можна додавати більше серверів за потреби).
- Вища надійність і відмовостійкість (відсутність єдиного точкового збою).
- Зниження вартості (використання дешевших серверів).

Недоліки:

- Складність реалізації (потрібні зміни в програмному забезпеченні для підтримки розподіленої архітектури).
- Потребує ефективного управління розподіленими системами.

Вертикальне масштабування підходить для систем з обмеженими вимогами до масштабованості і де важлива простота реалізації.

Горизонтальне масштабування підходить для великих систем з високими вимогами до масштабованості, надійності і продуктивності.

Яку базу даних обрати?

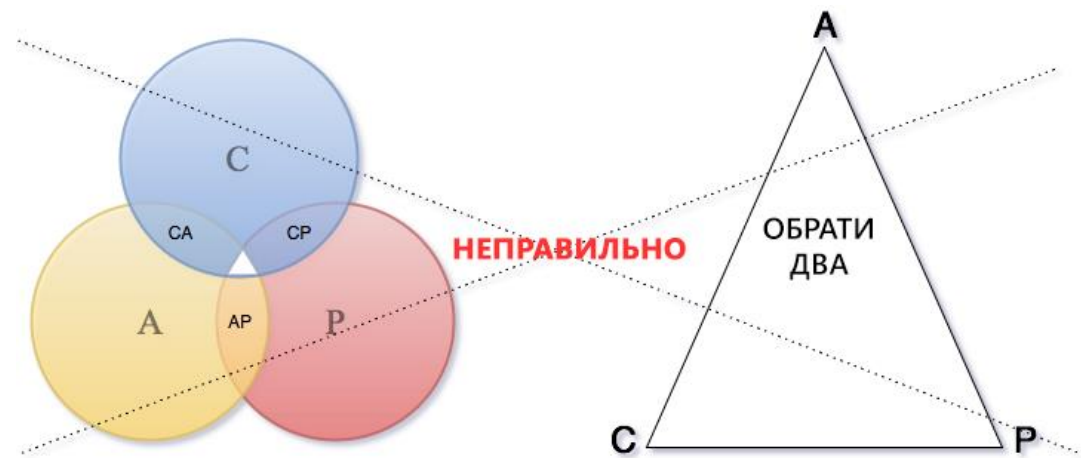
CAP-теорема

CAP-теорема (теорема Брюера) — це фундаментальний принцип розподілених систем, який стверджує: якщо сервери втрачають зв'язок між собою і стають ізольованими (**Partition**), то система фізично не здатна одночасно забезпечити сувору узгодженість даних (**Consistency**) та безперебійну доступність для користувачів (**Availability**).

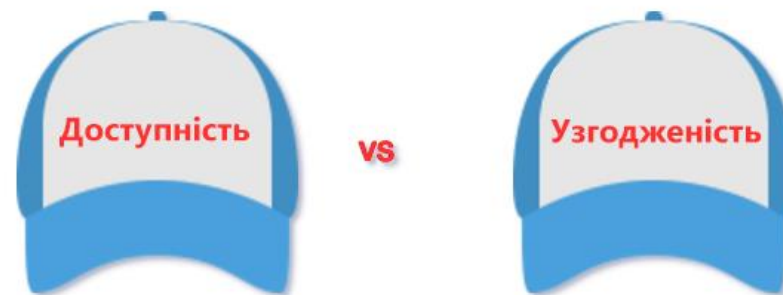
Consistency (узгодженість): у кожен момент часу всі вузли бачать абсолютно однакові дані. Будь-яка операція читання повертає найсвіжіший результат останнього запису.

Availability (доступність): кожен запит до працюючого вузла обов'язково отримує **успішну (без помилок)** відповідь, проте без гарантії, що вона містить найновіші дані.

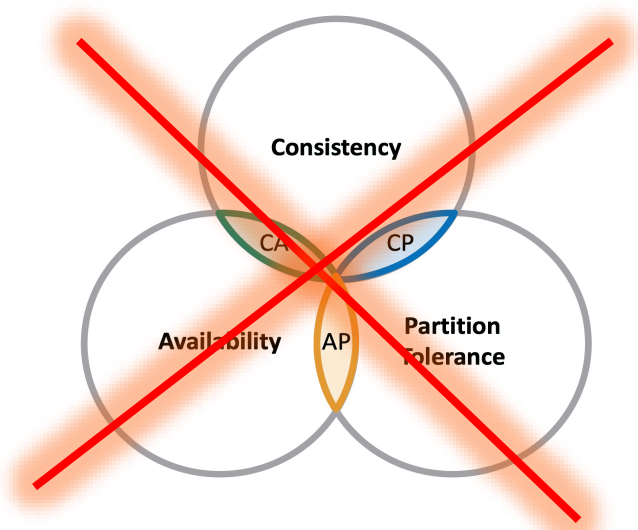
Partition Tolerance (стійкість до розділення): здатність системи продовжувати виконання запитів, навіть якщо зв'язок між окремими серверами (вузлами) тимчасово розірвано.



у разі мережевого розриву



CAP-теорема: Чому формула «Обери 2 з 3» — це міф?



У класичних підручниках часто стверджують, що можна обрати будь-які 2 властивості з 3. Проте в реальних розподілених системах це не так:

Чому комбінація СА неможлива?

У розподілених системах фізично неможливо побудувати комбінацію СА (Узгодженість + Доступність). Це утопічний сценарій, де система завжди має свіжі дані та працює без збоїв. Оскільки мережеві аварії все одно неминуче стануться, ми не можемо просто «вимкнути» фактор розщеплення (**P**).

Де насправді існує СА?

Лише на одному фізичному сервері (традиційні монолітні RDBMS на зразок класичного PostgreSQL або MySQL). Але один сервер за визначенням **не є розподіленою системою**.

Реальний вибір інженера:

Оскільки збої мережі є неминучими, у розподіленому кластері архітектор завжди обирає лише між двома моделями: **CP або AP**.

САР-теорема: Два практичні шляхи

Під час аварійного розриву мережі (Partition) архітектор системи змушений обрати один із двох фундаментальних пріоритетів:

Шлях CP (Пріоритет — Суворості точність даних)

Логіка: *«Краще ми покажемо помилку і заблокуємо дію користувача, ніж дозволимо використати неправильні чи застарілі дані».*

Під час аварії: Вузол, який втратив зв'язок із більшістю кластера, відхиляє запити, блокуючи оновлення даних до відновлення мережі.

Критично для: банківські перекази, списання балансу, купівля та бронювання квитків, зміна паролів.

Шлях AP (Пріоритет — Швидкість та безперебійність)

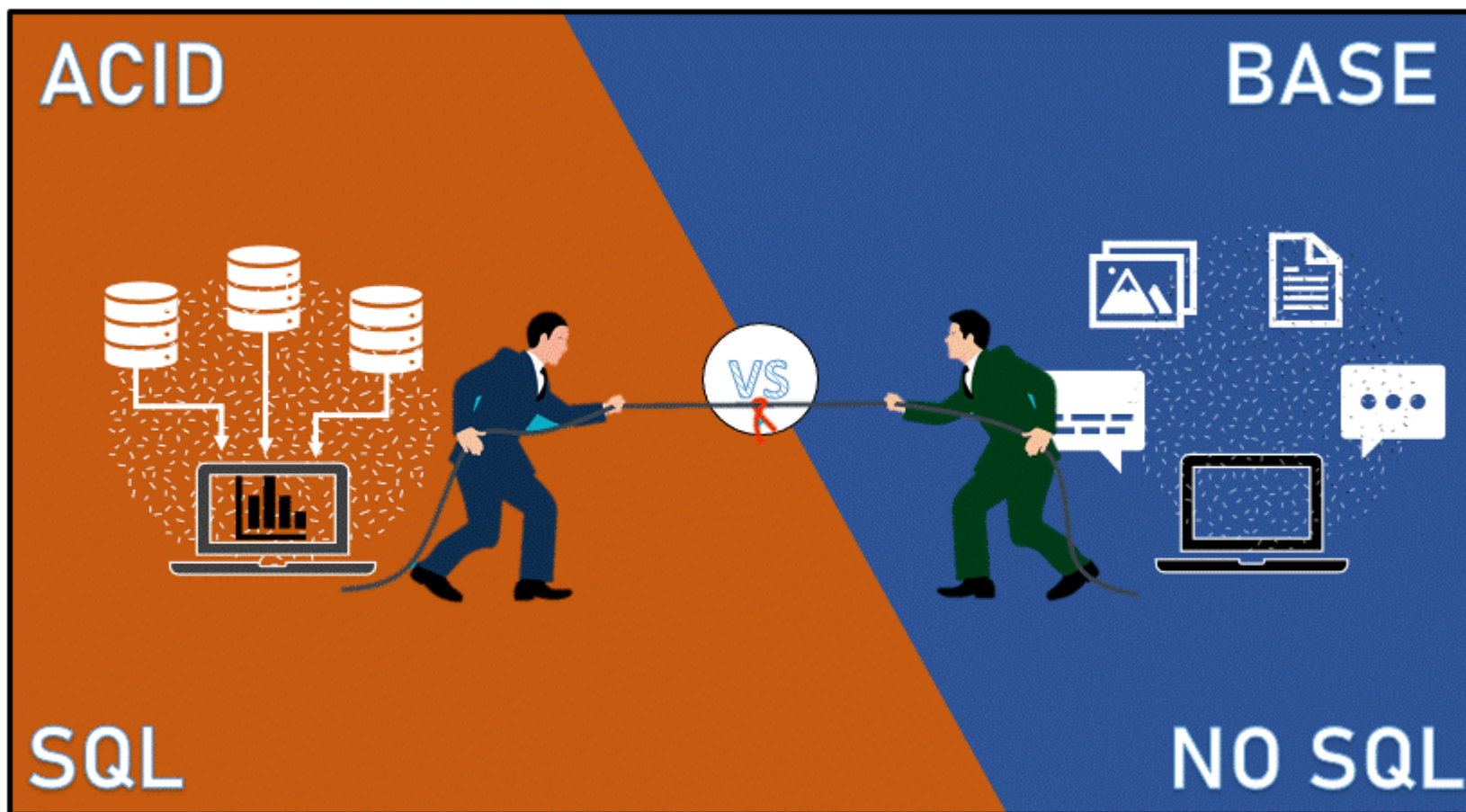
Логіка: *«Система повинна реагувати миттєво за будь-яких умов. Якщо дані трішки застаріли — не страшно, ми синхронізуємо їх у фоні після відновлення зв'язку».*

Під час аварії: Кожен доступний сервер самостійно приймає запити, зберігає їх локально та віддає наявні дані (навіть якщо вони не найсвіжіші).

Критично для: стрічка соцмереж, лічильники лайків, кошик покупок, аналітика та чати.

ACID vs BASE: дві філософії надійності

ACID і **BASE** — це дві різні моделі узгодженості, які використовуються в БД для забезпечення надійності та продуктивності.



ACID ≠ тільки реляційні БД, а BASE ≠ всі NoSQL.

Це радше типові тенденції.

ACID модель

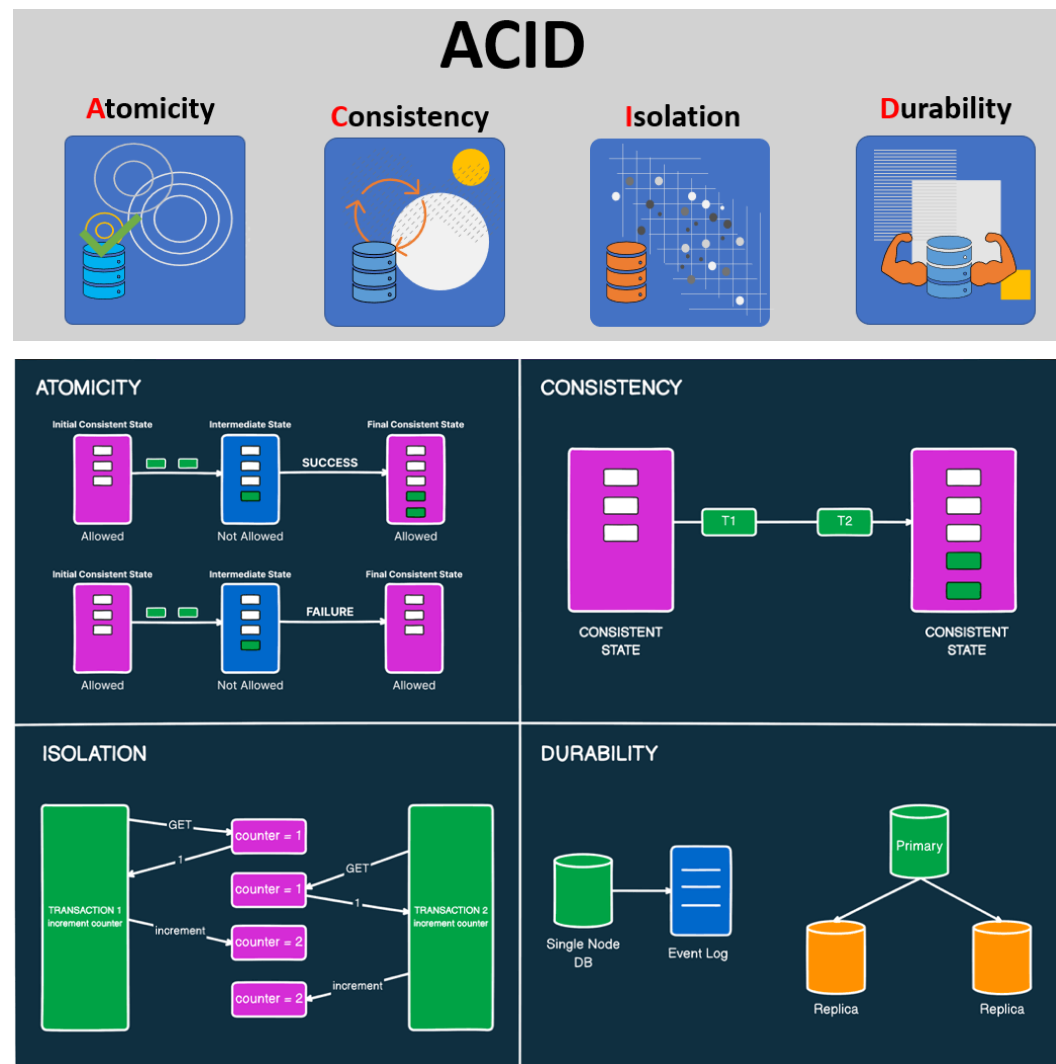
ACID — це набір властивостей, які гарантують надійність транзакцій у реляційних базах даних:

Атомарність (Atomicity): Транзакція є неподільною одиницею роботи, яка або виконується повністю, або не виконується взагалі. Якщо будь-яка частина транзакції не вдається, вся транзакція відміняється.

Узгодженість (Consistency): Після завершення транзакції всі дані в базі даних залишаються в правильному стані, дотримуючись усіх визначених правил, обмежень і тригерів. Якщо транзакція порушує будь-яке з цих правил, вона не буде завершена, і база даних повернеться до попереднього стану.

Ізольованість (Isolation): Транзакції виконуються незалежно одна від одної. Проміжні стани транзакції не видимі для інших транзакцій, що запобігає взаємному впливу транзакцій.

Стійкість (Durability): Після завершення транзакції її результати зберігаються постійно, навіть у разі збою системи. Це означає, що дані будуть збережені на диску або іншому надійному сховищі.



Транзакції забезпечують несуперечливість даних у реляційних БД. Вони гарантують виконання всіх команд або їх повне скасування у разі помилки, зберігаючи цілісність даних.

BASE модель

BASE — це набір принципів, які часто використовуються в NoSQL базах даних для забезпечення гнучкості та масштабованості:

Базова доступність (Basically Available): Система завжди доступна для обробки запитів, навіть якщо деякі частини системи не працюють. Це означає, що система може відповідати на запити, але не обов'язково з найсвіжішими даними.

Слабка узгодженість (Soft state): Стан системи може змінюватися з часом, навіть без нових запитів, через реплікацію і синхронізацію даних між вузлами. Це означає, що дані можуть бути тимчасово неузгодженими.

Кінцева узгодженість (Eventual consistency):

З часом всі вузли системи досягнуть узгодженого стану, але не обов'язково миттєво. Це означає, що дані в кінцевому підсумку стануть узгодженими.



ACID vs BASE: Порівняння філософій



- **Песимістичний підхід**
передбачає блокування і жорсткий контроль, щоб уникнути помилок
- **Сильна узгодженість**
дані завжди правильні та актуальні
- **Пріоритет — Точність**
гарантія правильності важливіша за швидкість
- **Складно змінювати - Жорстка схема (Schema-on-write)**
у реляційних БД структура таблиць жорстко визначена
- **Може бути доступним**
система може бути доступною, але це не завжди головний пріоритет (узгодженість часто важливіша)
- **Масштабування — Вертикальне (Scale-Up)**
нарощування потужності одного сервера, складніший розподіл на кластер.



- **Оптимістичний підхід**
дозволяє операціям виконуватися паралельно, а можливі конфлікти вирішуються після запису
- **Слабка узгодженість**
допускається, що деякі дані можуть бути застарілими чи неповними
- **Пріоритет — Доступність (приблизність прийнятна)**
точність жертвують заради швидкості та доступності
- **Легко змінювати - Гнучка схема (Schema-flexible)**
NoSQL часто не має жорсткої схеми, що робить модифікації простішими
- **Має бути доступним**
доступність ставиться у пріоритет вище за узгодженість
- **Масштабування — Горизонтальне (Scale-Out)**
Нативна робота на великих кластерах із десятків звичайних серверів.

Порівняння реляційних та нереляційних баз даних

Основні переваги реляційних БД:

- Структурованість і організованість
- Мова запитів SQL
- Цілісність даних
- Нормалізація
- Транзакції
- Безпека
- Масштабованість (обмежена)
- Підтримка складних запитів і звітів
- Зрілість і надійність
- Широка підтримка і спільнота

Основні переваги NoSQL БД:

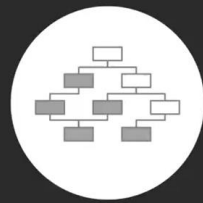
- Гнучкість у моделюванні даних
- Масштабованість
- Висока продуктивність
- Стійкість до розділення
- Підтримка неструктурованих і напівструктурованих даних
- Різноманітність типів баз даних
- Простота розробки і розгортання
- Висока доступність і відмовостійкість



Banking Systems



Accounting



Complex SQL Queries



Smaller Systems



Caching



User Sessions



Real-Time Inventory



Fraud Mitigation

Порівняння реляційних та нереляційних баз даних

Наскільки доцільно використовувати NoSQL або реляційну базу даних для вашого проекту з точки зору часу, грошей і технологій?

Час:

SQL: Може вимагати більше часу на початкове проектування схеми даних, але добре підходить для проєктів з чітко визначеними вимогами.

NoSQL: Може бути швидше в налаштуванні для проєктів з гнучкими або змінними вимогами до даних.

Гроші:

SQL: Може бути дорожчим у масштабуванні через необхідність вертикального масштабування.

NoSQL: Може бути економічно вигіднішим для великих обсягів даних і високих навантажень завдяки горизонтальному масштабуванню.

Технології:

SQL: Добре підтримується багатьма інструментами і фреймворками, має велику спільноту і документацію.

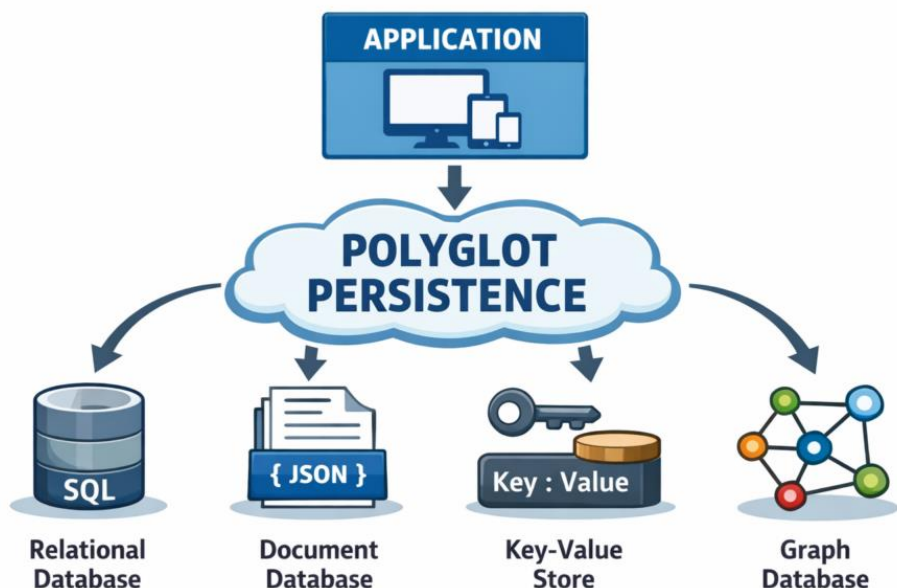
NoSQL: Підходить для сучасних веб-додатків, великих даних, реального часу і аналітики.

Сучасний стандарт: Polyglot Persistence

Сучасна архітектура більше не обирає «тільки SQL» або «тільки NoSQL».

Вона часто керується принципом **багатоваріантної персистентності (Polyglot Persistence)**:

використання різних типів СУБД для вирішення різних за специфікою задач у межах одного проєкту



Наприклад, сучасна e-commerce платформа може використовувати:

- **Реляційну БД (PostgreSQL):** для фінансових транзакцій та оформлення замовлень (де необхідні ACID-гарантії);
- **Документо-орієнтовану БД (MongoDB):** для гнучкого каталогу товарів із різноманітними характеристиками;
- **БД типу Ключ-Значення (Redis):** для кешування сесій користувачів та збереження кошика покупок;
- **Графову БД (Neo4j):** для рекомендаційної системи («з цим товаром також купують»);
- **Векторну БД (Pinecone):** для інтелектуального семантичного пошуку товарів, пошуку за фотографією та роботи AI-асистента магазину.

Використання та застосування нереляційних баз даних



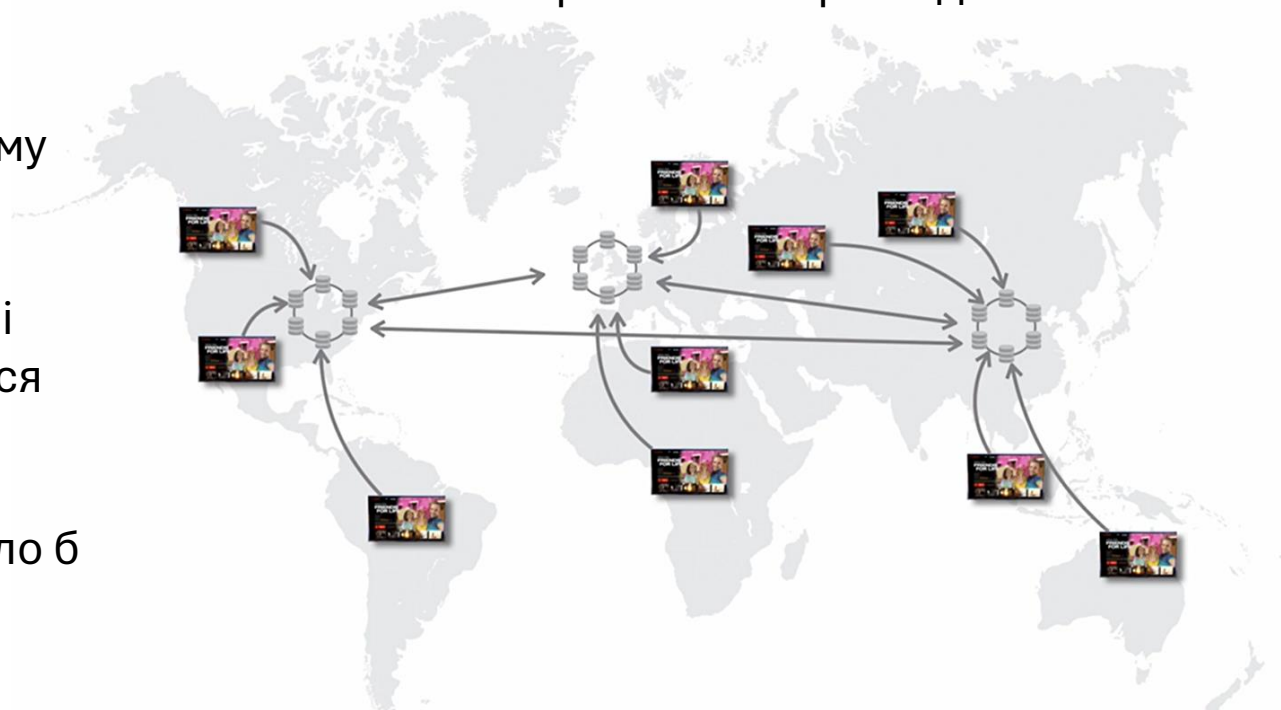
NETFLIX

Сервіси на кшталт Netflix використовують масштабованість, продуктивність і надійність NoSQL баз даних, щоб забезпечити майже миттєвий і безперебійний перегляд.

Netflix одночасно дивляться мільйони людей по всьому світу, і всі ці взаємодії зберігаються за мілісекунди.

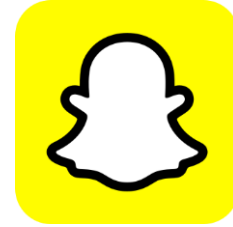
Кожного разу, коли ви дивитеся шоу, збираються дані про те, що ви переглядаєте, як довго, де ви зупинилися тощо, і ці дані використовуються для рекомендацій

До появи NoSQL баз даних надання такого сервісу було б надзвичайно дорогим, якщо взагалі можливим.



NoSQL у великих технологічних компаніях

NETFLIX



Uber



ebay

LinkedIn

airbnb

amazon



Spotify

TINDER



Google

Pinterest



reddit

ДЯКУЮ ЗА УВАГУ!